

FINAL PROJECT REPORT

nosec is novuln?

Analyzing the Bandit exclusions in Python codebases to discover vulnerable patterns

Report by - Karthik, Sujeeth, Ankita, Manasi

Abstract- This project centers on a comprehensive analysis of Bandit exclusions within open-source Python packages hosted on PyPi repositories. The study involves examining the prevalence, distribution, and patterns of these exclusions to understand their impact on security evaluations better. Through extensive research, we aimed to identify common coding practices or functions often exempted from security checks using the *nosec* annotation. The investigation sought to unveil the frequency and nature of exclusions applied, enabling insights into potential security loopholes and areas requiring improved scrutiny in Python codebases.

I. Introduction

In the rapidly evolving landscape of software development, ensuring robust security measures within applications stands as a paramount concern. Static analysis tools such as Bandit serve as indispensable assets, spotlighting potential security vulnerabilities in codebases at an early developmental stage. Bandit, a prominent Static Application Security Testing (SAST) tool within the Python ecosystem, diligently scans code, flagging vulnerable patterns and generating warnings. However, software engineers often resort to Bandit exclusions, identified by the *nosec* annotation, to

sidestep specific warnings. This research embarks on an in-depth analysis of these exclusions found in Python packages hosted on PyPi repositories. By scrutinizing their prevalence, variations, and contextual usage, this study aims to uncover possible security oversights and promote best practices in handling Bandit warnings. We also demonstrate instances where these exclusions are vulnerable. Ultimately, this investigation seeks to refine our comprehension of employing exclusions judiciously, striking a balance between vigilant security measures and efficient code maintenance.

II. Literature review

Developers use Automatic Static Analysis Tools (ASAT) at different stages of development but often do not customize warnings for specific contexts [1]. This might lead to warnings being ignored based on the development context. In the study [2], it was discovered that a small fraction (8.76%) of reported issues were addressed, with 80% of the fixes concentrating on just 20% of the ASAT rules. Study [5] examined 20 Java open-source projects, revealing that while most reported issues were resolved, some were deliberately ignored through configuration changes. The research seeks parallels in the handling of security analyzers despite not directly targeting them.

Study [3] mined ASAT-reported violations, finding few fixed due to high false positive rates or developer disinterest in lower severity bugs. Our research aims to analyze ignored issues, aiming to automatically detect and report them while estimating the percentage of false positives among the ignored problems. Study [4] highlighted how a vulnerability in one Python package can affect numerous dependent packages, emphasizing the criticality of securing both the codebase and its dependencies, with an average of 250 packages relying on the top 10,000 PyPi packages (Package Reach).

Study [12] lists prevalent vulnerabilities in Python packages, emphasizing the necessity for continuous detection, while our focus extends to identifying vulnerabilities in assumed safe Python repositories by examining code sections marked for exceptions using *nosec*.

Paper [15] centers on a Python tool utilizing SAST, employing an Abstract Syntax Tree (AST) and data flow relationships to detect vulnerabilities via a plugin-based architecture, offering flexibility without external dependencies for rule modification or creation.

III. Objective

1. Analyze Bandit Exclusions:

Review code repositories using Bandit, particularly focusing on sections where *nosec* comments or exclusions are applied.

Identify the specific types of security warnings or vulnerabilities that have been excluded using these annotations.

2. Identify Patterns Leading to Exclusions:

Examine the context around the flagged code segments. Look for recurring patterns

that lead developers to add *nosec* exclusions. Common reasons for exclusion might include false positives, lack of understanding of the vulnerability, or deadline pressures. Assess if the code follows certain risky practices or design patterns that trigger Bandit warnings.

3. Review Ignored Vulnerabilities:

Cross-reference Bandit's flagged vulnerabilities with the code where *nosec* annotations are applied. Verify if any of these ignored vulnerabilities are actual security threats and not false positives. Assess the impact these vulnerabilities might have if exploited.

4. Propose Improvements:

Educate developers about the nature of these vulnerabilities to reduce false positives and enhance understanding. Implement a systematic process for reviewing and addressing Bandit findings to distinguish genuine threats from false positives. Encourage a culture of security awareness and proactive mitigation strategies rather than reactive *nosec* additions.

5. Implement Changes:

Integrate best practices into the development workflow to avoid the recurrence of similar Bandit exclusions. Regularly update and fine-tune Bandit configurations to reduce false positives and improve its effectiveness in identifying genuine vulnerabilities.

IV. Why Bandit?

1. *Static Application Security Testing (SAST)*: Bandit is categorized as a SAST tool, which means it analyzes code statically, identifying security weaknesses and vulnerabilities in the codebase.

2. *Plugin Support*: Bandit is flexible and extensible. It supports the integration of plugins, allowing users to extend the tool's default set of checks and add custom checks tailored to specific requirements.

3. *Pre-built Checks*: It offers a collection of out-of-the-box checks that target common insecure coding practices and potential vulnerabilities commonly found in Python applications. These checks cover various security issues and are continuously updated to adapt to evolving threats.

4. *Identifying Vulnerabilities*: Bandit is particularly adept at flagging patterns and coding practices that might lead to security vulnerabilities, such as SQL injection, code injection, hard-coded secrets, and more.

V. Bandit Security Checks

Bandit covers a spectrum of security concerns including hardcoded passwords, insecure function usage, code injection risks like SQL or OS command injections, insecure cryptographic practices, unreliable randomness, flawed file permission settings, inadvertent exposure of sensitive information like API keys or passwords etc. By meticulously scanning codebases, Bandit assists in identifying and rectifying these security pitfalls, aiming to fortify the overall security posture of Python applications. Bandit's comprehensive analysis helps developers fortify their code against common security flaws, ensuring robust protection against potential exploits.

VI. Exclusions in Bandit

When Bandit encounters a nosec comment, it recognizes this as an indication to bypass

the security warning and not report it as an issue.

Developers might add nosec comments for various reasons, such as when:

1. The flagged security issue is a false positive, meaning Bandit incorrectly identified a piece of code as vulnerable or risky. This is a common behavior, as static tools are designed to be sound rather than complete.

2. The identified vulnerability is known but has been assessed and deemed acceptable within the context of the code's usage or specific requirements.

3. There's a critical reason for keeping the code as-is despite the flagged vulnerability (e.g., due to compatibility constraints or certain design decisions).

```
self.process =  
subprocess.Popen('/bin/echo', shell=True)  
# nosec
```

Table 1

VII. Why did we study Bandit Exclusions?

Studying Bandit exclusions was necessitated by their widespread use within codebases. Figure 1 shows the spread of usage among repositories. These exclusions, often denoted by the nosec annotation, are employed to bypass security warnings flagged by Bandit. The sheer volume of these exclusions across various Python packages hosted on repositories like PyPi prompted our investigation. This scrutiny aimed to understand the reasons behind their prevalence and the potential security implications associated with their usage. For

instance, considering an argument such as `http_response` originating from a remote server, especially if it can be influenced or controlled by a malicious actor, raises concerns about potential vulnerabilities within the codebase. The examination of Bandit exclusions seeks to uncover scenarios where security risks might be overlooked or where certain code patterns could pose substantial threats if not properly addressed.

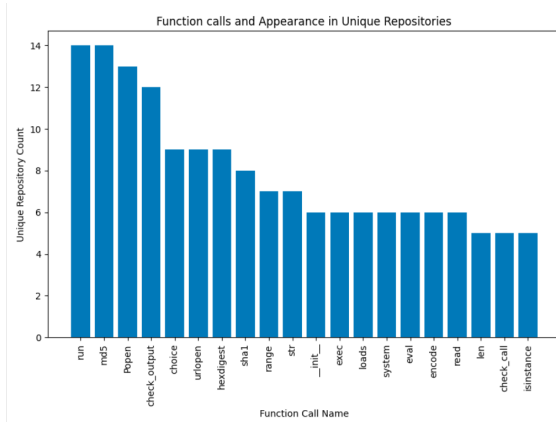


Figure-1

VIII. Methodology

A. Scraping Pypi GitHub Repositories

To make sure we select a good set of packages to analyze, we picked the data set with the top 8000 PyPi packages [20]. To further filter only the packages that made use of Bandit in the development or build system, we made use of [21] GitHub REST APIs. This filtering is done based on a heuristic to determine the presence of Bandit usage. Once the set of repositories was determined, text analysis was performed on the source code to identify all the presence of nosec exclusions.

B. Analyzing Nosec Instances

After scraping the Github repositories we analyzed all the nosec instances. We used regular expressions to match the library imports and function calls and extracted the nosec codes used to look for patterns.

Figure 1 indicates the frequency of functions used in the nosec instances we found. It was evident that the *eval* function had the highest frequency of nosec statements, signifying that this function was frequently used with exclusions for security checks. Following *eval*, cryptographic methods like *sha1* and *md5*, as well as system interaction functions like *popen* and *run*, also appeared with notable frequency in the nosec exclusions. While, the analysis of nosec appearances within PyPi packages reveals that the function *run* has the most frequent occurrences of exclusion annotations, closely trailed by *md5*, *popen*, *check_output*, *choice*, *urlopen*, *hexdigest*, *sha1*, and *range*, among others.

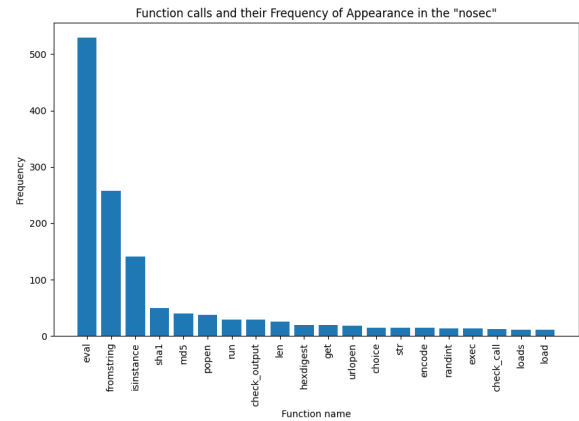


Figure-2

From the 8000 PyPi repositories we analyzed, particularly focusing on the usage of Bandit and its exclusions, revealed some notable statistics:

1. Bandit Usage: Out of the 8000 repositories analyzed, 282 used Bandit for

security testing (3.5%). While it's a positive sign that a portion of projects are employing a security-focused tool, the number could be higher considering the importance of security in software development. This could also suggest the prevalence of other static analyzers such as *semgrep*, *snyk*, etc.

2. Presence of nsec: Among the repos utilizing Bandit, 162 (57.4%) had at least one nsec exclusion added. This indicates either false positives, acceptance of risks, or overlooked security warnings.

3. Category in Nsec Exclusions: Surprisingly, only 7.34% of the nsec exclusions had the category of exclusion specified, suggesting that most exclusions lack proper categorization or details regarding the reason for the exclusion.

IX. Attacking Potential Vulnerabilities

A. DBND Library [18]

DBND is an open-source framework for building and tracking data pipelines. dbnd-docker supports building docker images.

```
cmd = "docker push {}".format(self.image_name_with_tag)
logger.info("Running docker push command: '%s'", cmd)
run_cmd(cmd, shell=True)
```

Figure-3

There is an instance in this library that takes the `image_name` parameter in the `docker_build.py` file within the `dbnd-docker` as a user input or configuration from a non-privileged user and is not properly sanitized. Figure 3 shows the part of the code where this occurs. In that case, it poses a risk of command injection. Since `shell=True` is enabled, passing arbitrary commands within the `image_name`

parameter could allow an attacker to execute additional commands within the Docker environment.

```
(.venv) ~ - dbnd exec /tmp/demo
(.venv) ~ - dbnd docker run dbnd_docker.docker.docker_build.DockerBuild --verbose --set image_name=test | touch /tmp/demo/exploited; # 2> /dev/null
(.venv) ~ - dbnd ls /tmp/demo
exploited
```

Figure-4

Injecting a command like `touch`(Figure-4) demonstrates that arbitrary commands can be passed through the `image_name` parameter.

```
sp = subprocess.Popen(
    cmd,
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    bufsize=1,
    universal_newlines=True,
    shell=shell, # nsec B602 (Ignore this for bandit as this is code used for orchestration)
    **kwargs
)
```

Figure-5

If the Docker instance runs under a privileged user, this vulnerability could be exploited for privilege escalation. As a result, it's crucial to ensure proper input validation and sanitation of any user-controlled inputs, especially those used to construct system commands or interact with subprocesses like Docker commands, to prevent such command injection vulnerabilities.

B. newrelic-python-agent [19]

Using `eval` in the context of processing code that starts with a `lambda` within the `newrelic-python-agent` package's `config.py` (Figure-6) file could potentially introduce a vulnerability.

```
1347 if name and name.startswith("lambda "):
1348     callable_vars = ("callable_name": newrelic.api.object_wrapper.callable_name)
*** 1349     name = eval(name, callable_vars) # nsec, pylint: disable=E0121
```

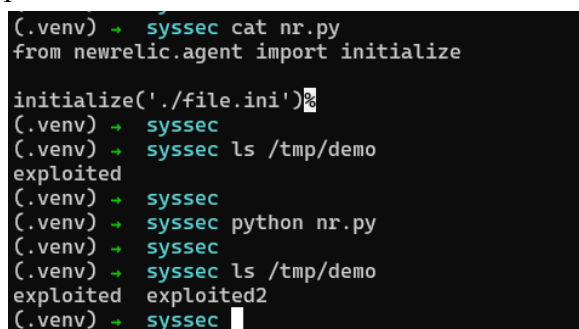
Figure-6

```
(.venv) ~ - syssec cat file.ini
[database-trace:test]
enabled = true
sql = lambda x: x.__import__('os').system('touch /tmp/demo/exploited2')
function = hello:world
```

Figure-7

When the application evaluates code dynamically using `eval`, especially if it's based on user-controlled or configurable input, it poses a risk of code execution vulnerabilities. Crafted configurations or inputs could contain malicious code that, when processed by `eval`, might lead to unauthorized actions or possibly the execution of arbitrary code.

The severity of this vulnerability depends on the deployment and how the configuration is managed. If the configuration can be manipulated or controlled by an unauthorized or lower-privileged user, it could result in privilege escalation or unauthorized code execution within the context of the application, leading to a potential security breach. Figure 7 shows an example configuration file that is being used to show the exploit, you can see that we have a Python code starting with `Lambda` but also runs an `os.system` command leading to a code injection attack (Figure-8). Therefore, it's crucial to review and ensure that user inputs or configurable settings undergo proper validation and sanitation to prevent such risks.



```
(.venv) ➔ syssec cat nr.py
from newrelic.agent import initialize

initialize('./file.ini')
(.venv) ➔ syssec
(.venv) ➔ syssec ls /tmp/demo
exploited
(.venv) ➔ syssec
(.venv) ➔ syssec python nr.py
(.venv) ➔ syssec
(.venv) ➔ syssec ls /tmp/demo
exploited exploited2
(.venv) ➔ syssec
```

Figure-8

X. Nosec exclusion usage observations

Based on our observations, the pattern of

nosec usage can be grouped into the following categories

1. Unsafe exclusions: Cases where the source code exhibits obviously vulnerable behavior fall under this category. One such example is passing user-controlled input into `os.system` function calls. This type of exclusion is the primary focus of our study. We found a very small number of such instances and were able to exploit them in the previous section.

2. Maybe-safe exclusions: This includes cases where insecure behavior is minimized using some form of custom sandboxing. One such example we observed was validating the string against common exploit payloads like underscores and filtering them before passing it to the `eval` function. This might look safe in a code review but does not promise the absence of a vulnerability. Best practices often suggest not including such hand-crafted solutions to sandbox insecure behavior. These instances would need further analysis and a better understanding of the codebase to explore the exploitability. This also suggests opportunities to use better bug-finding techniques like fuzzing, symbolic execution and dynamic taint analysis. There is a large area and attack vector to explore here.

3. Safe exclusions: This includes usages where no user-controlled input flows into the code or the user has no control over the behavior. Common instances we observed were running `subprocess.Popen` with hardcoded only arguments, adding *nosec* exclusions in the test code etc. These instances are easy to identify from code reviews and can be filtered out of further analysis.

The following table depicts our observations of exclusion categories in the packages analyzed. Our study focused on the Unsafe category which is significantly small. Even though the majority of the usages are safe, there is still a large part of the maybe-safe exclusions that need to be studied to identify possible vulnerabilities.

Exclusion category	Percentage of <i>nosec</i> occurrences
Safe	84.06%
Maybe-safe	12.18%
Unsafe	3.55%

Table 2

XI. Challenges:

The study of Bandit exclusions encountered several challenges. The foremost among these was the arduous nature of manually analyzing exclusions within codebases, especially in larger repositories, demanding substantial time and effort. Identifying viable solutions for future improvements emerged as an ongoing task, requiring thorough exploration and implementation considerations. Another significant challenge involved efficiently pinpointing and filtering repositories, hindered by limitations within the GitHub graph API, particularly the absence of robust code filtering capabilities and imposed rate limits. Additionally, the reliance on heuristics to detect Bandit usage might lead to false positives, necessitating more precise detection methodologies for accurate identification and evaluation of code

repositories. Addressing these challenges remains pivotal to enhancing the effectiveness of Bandit exclusion analysis within open-source projects.

XII. Future Work

Future work aims to streamline the analysis of Bandit exclusions by developing an automated tool capable of generating the call path leading to the *nosec* line. This tool would significantly accelerate the analysis process while effectively filtering out safe exclusions. The concept revolves around generating a call graph for a specified module and subsequently refining the paths that culminate in the target line. Moreover, incorporating taint analysis techniques would aid in identifying user-controlled input along these paths. Focusing on possibly safe exclusions, the integration of fuzzing methodologies is planned to enhance the identification of potential bugs. Expansion beyond Python and Bandit is also a priority, to encompass other static analysis tools and languages to offer comprehensive coverage. Additionally, plans involve executing Bandit on projects not currently utilizing security analyzers to broaden the scope of vulnerability detection across diverse codebases.

XIII. Conclusion

The presence of *nosec* instances alone doesn't guarantee code safety. Bandit exclusions, denoted by *nosec*, require detailed explanations and validation by developers because they might not universally signify secure practices. Despite the widespread use of security analyzers in open-source projects, they demand careful

examination. Developers play a crucial role in providing clear explanations and justifications for using exclusions. This practice ensures that potential vulnerabilities aren't simply suppressed but are genuinely identified, addressed, and resolved. Continuous vigilance, thorough explanations, and ongoing evaluation remain essential elements for upholding strong security standards in software development.

IX. References

1. Vassallo, C., Panichella, S., Palomba, F., Proksch, S., Zaidman, A., & Gall, H. C. (2018). *Context is king: The developer perspective on the usage of static analysis tools*
2. Marcilio, D., Bonifácio, R., Monteiro, E., Canedo, E., Luz, W., & Pinto, G. (2019). *Are Static Analysis Violations Really Fixed? A Closer Look at Realistic Usage of SonarQube*.
3. Liu, K., Kim, D., Bissyandé, T. F., Yoo, S., & Traon, Y. L. (2019). *Mining Fix Patterns for FindBugs Violations*.
4. Bagmar, A., Wedgwood, J., Levin, D., & Purtilo, J. (2021). *I Know What You Imported Last Summer: A study of security threats in the Python ecosystem*.
5. Zampetti, F., Scalabrino, S., Oliveto, R., Canfora, G., & Penta, M. D. (2017). *How Open Source Projects Use Static Code Analysis Tools in Continuous Integration Pipelines*.
6. Nascimento, A. (2020). [Ensure that each nosec usage has an accompanying explanation · Issue #478 · PyCQA/bandit](#)
7. Kuszczynski, K. & Walkowski, M. (2023). *Comparative Analysis of Open-Source Tools for Conducting Static Code Analysis*
8. Lenarduzzi, V., Pecorell, F., Saarimaki, N., Lujan, S., & Palomba, F. (2023). *A critical comparison on six static analysis tools: Detection, agreement, and precision*
9. Stefanović, D., Nikolić, D., Havzi, S., Lolić, T., & Dakić, D. (2021). *Identification of strategies over tools for static code analysis*
10. Paramitha, R., & Massacci, F. (2023). *Technical leverage analysis in the Python ecosystem*
11. Alfadel, M., Costa, D., & Shihab, E. (2023). *Empirical analysis of security vulnerabilities in Python packages*
12. Kapustin, D. A., Shvyrov, V. V. & Shulika, T. I. (2023). *Static Analysis of Corpus of Source Codes of Python Applications*
13. Beller, M., Bholanath, R., McIntosh, R., Zaidman, A., (2016). *Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software*
14. Ruohonen, J., Hjerpe, K., & Rindell, K. (2012). *A Large-Scale Security-Oriented Static Analysis of Python Packages in PyPI*
15. Ma, L., Yang, H., Xu, J., Yang, Z., Lao, Q., & Yuan, D. (2022). *Code Analysis with Static Application Security Testing for Python Program*
16. Ayewah, N., Pugh, W., Hovemeyer, D., Morgenthaler, J. D., & Penix, J. (2008). *Using Static Analysis to Find Bugs*

17. Ayewah, N., Pugh, W., Morgenthaler, J. D., Penix, J. & Zhou, Y. *Evaluating Static Analysis Defect Warnings on Production Software*

18. [Github - DBND](#)

19. [GitHub - New Relic Python Agent](#)

20. [Top PyPI Packages: A monthly dump of the 8,000 most-downloaded packages from PyPI \(hugovk.github.io\)](#)

21. [GitHub REST API documentation - GitHub Docs](#)